NLP homework 1

Goryunov Kirill
Higher School of Economics, Faculty of Computer Science
(Dated: April 15, 2026)

The source code of the project can be viewed on github: https://github.com/randnull/nlp

I. INTRODUCTION

This work aims to research an tokenization and BPE algorighms and compare different approaches to them.

II. DATASET

Our dataset for this task will be imdb-dataset-of-50k-movie-reviews, that contains more than 50.000 movie reviews. We will use it as one big text, just union it.

III. NORMALIZATION AND TOKENIZATION

To tokenize text we have a several options, including use "split" by whitespace methods and regular expression. Regular expression is more usefull due to it found words with different separators, like dots and whitespaces, when original split use only whitespaces.

So, we will use regular expression tokenization:

```
(?:\w+')\w+|(?:[A-Z]\.)+|\w+(?:-\w+)*|[\w+\.]
```

But this method's output give a list of tokens without any normalization, for example:

`tokenize("I love you.NLP")` ⇒ `['I', 'love', 'you', '.', 'NLP']`.

Before tokenization we can do some base operations: convert to lowercase.

After tokenization we also can apply some normalization methods, for example: lemmatization and stemming, where we delete word's ending (running -> run). But the main goal of this task is to create BPE algorigtm, that main purpose to do the same.

IV. BYTE PAIR ENCODING

The main point of the algorithm is to find a balance between using whole words and letters of the alphabet.

If we use whole words, we have troubles with size of our vocabulary. For english language we have estimation of unique number of words between 500.000 and 1.000.000. This number not contains different form of words (like 'ing', 'able'), so the count will be more and more bigger. This method has advantages in the length of the word vector representation. This makes it easier to learn any embeddings for words, build sentences and learn models, because each word is a unique object.

As for the letters of the alphabet for word representetion we also have several advantages and disadvantages. Our dictionary size no more than 100-200 in main cases (alphabet + ascii). But every word is just combination of independent indexes from alphabet.

BPE is the middle ground between them. The goal of the algorithm is to start with all the symbols being separate and compress the repeating ones together as much as possible, that we can see in 1.

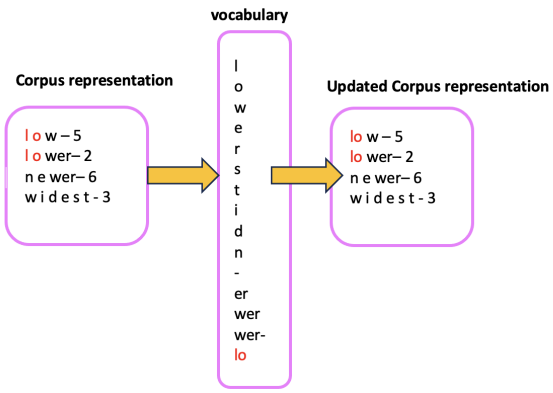

FIG. 1: Byte Pair Encoding

Next step is run algoritm in python and check the results.

V. RESULTS

In the beginning we only had 71 words in the dictionary (just an alphabet). After 300 merges we get 346 words. For comparison, if we considered each word unique, then the size of the dictionary would be 3274.

Now let's see in the words:

```
look  that  most  other  ing  ars  ever  into
_which  was  are  _int  ves  ide  bet  _ro
have  ment  end  _other  would  who  com  any
```

BPE successfully found classical endings ('ing', 'ment'), prepositions and connectives (are, any, into).

This will obviously help for the formation of tokens, since now each new word will consist of parts of this dictionary. (like in the article: [1])

[1] Kaggle, *huggingface, BPE*, https://huggingface.co/learn/llm-course/ru/chapter6/5) (accessed: 15 April 2026).